

Why INT8? — Summary of Evidence (Float32 vs INT8, and where INT4 fits)

This document answers one question directly: **why did we choose INT8 for the FPGA implementation, instead of staying with Float32 or going further to INT4?**

There are two separate pieces of evidence, answering two separate questions. Both are needed together to justify the choice.

1. **Software evidence** — does the trained network still classify digits correctly if we round its weights down to INT8? (Tested in Python, no hardware involved.)
 2. **Hardware evidence** — does building the FPGA circuit in INT8 actually make it smaller/faster than building it in Float32? (Tested by actually synthesizing both versions in Vitis HLS and reading the tool's own reports.)
-

Part 1 — Software Evidence: Does Accuracy Survive Quantization?

What we did: took our trained LeNet-5 model (float32 weights, ~98.4% test accuracy), and simulated what would happen to its accuracy if the weights and activations were rounded to lower-precision number formats — without touching any hardware, purely in Python/NumPy. Tested on 500 random MNIST test images.

Format	Test Accuracy	Max Weight Error	Memory Needed
Float32 (original, no quantization)	98.40%	0 (reference)	241.0 KB
Fixed16 <13, 3> (the paper's own format)	97.20%	0.0625	120.5 KB
INT8 (our choice)	98.40%	0.0024	60.3 KB
INT4	97.80%	0.0438	30.1 KB
Ultra-low (1-bit integer, 2-bit fraction — deliberately extreme test)	69.60%–95.40%	0.125	30.1 KB

Conclusion from this table: INT8 loses **zero accuracy** compared to Float32 (both 98.40%) while using **only 25% of the memory**. INT4 is smaller still, but starts to lose a small amount of accuracy (97.80% vs 98.40%). The paper's own fixed16 format actually does *worse* than our INT8, both in accuracy and in memory — because INT8 uses a scale factor calculated from each layer's real data range, while fixed16 uses one fixed format for everything.

This is why INT8, not INT4: INT8 gives the same accuracy as full Float32 at a quarter of the memory cost — a "free" improvement with no measured downside. INT4 would save a little more memory but starts to cost real accuracy, which is not a trade worth making when INT8 already achieves the memory savings we needed.

(Source: Results/quantization_comparison.csv, quantization_comparison_relu.csv, combined_comparison.csv, and accuracy_comparison_plot.png — all built and verified in Task 12-13 of the software phase, before any hardware work began.)

Part 2 — Hardware Evidence: Does INT8 Actually Improve the Real Circuit?

What we did: for every one of the seven layers in our LeNet-5 accelerator, we built **two real, separately synthesized hardware designs** — one using standard 32-bit floating-point arithmetic (`float`), and one using genuine 8-bit integer arithmetic (`int8` data, `int32` accumulation) — and compared the **actual synthesis reports** produced by Vitis HLS for both. These are not estimates or simulations: they are the real tool output describing what each design would actually need if built.

Correctness was checked before any of these numbers were trusted: every INT8 layer's output was verified to match a Python reference calculation **exactly**, value-for-value, with zero mismatches — a stricter standard than used anywhere else in this project, appropriate for integer arithmetic (which has no legitimate excuse for even the smallest difference, unlike floating-point rounding).

Per-Layer Hardware Comparison (Float32 → INT8)

Layer	LUT Change	FF Change	DSP Change	Latency Change	Timing (100MHz target)
C1 (1st Convolution)	−18%	−66%	−21%	−14% (faster)	Failed → Met
C3 (2nd Convolution)	−21%	−49%	−25%	−6% (faster)	Failed → Met
S2 (1st Pooling)	−37%	−73%	0% (already 0)	~unchanged	Met → Met
S4 (2nd Pooling)	−40%	−74%	0% (already 0)	~unchanged	Met → Met
C5 (1st Dense)	−19%	−60%	0% (unchanged)	−46% (faster)	Failed → Met
F6 (2nd Dense)	−12%	−47%	0% (unchanged)	−36% (faster)	Failed → Met

Output (Final/ Softmax)	-20%	-34%	+14% (see note below)	-32% (faster)	Failed → Met
-------------------------------	------	------	--------------------------	------------------	------------------------

Note on the Output layer's DSP increase: this is the one layer where DSP usage went up (14 → 16) rather than down. Traced directly through the actual synthesis reports (not just the summary numbers), this breaks into three precise pieces, not one:

- **A real saving** in the main calculation stage (5 → 4 DSP) — smaller, narrower INT8 multiplication genuinely needs less hardware than Float32, exactly as in every other layer.
- **An accounting relocation, not new work** (an existing adder's cost moved from "shared with another part of the design" to "counted locally," because the Float32 version had another floating-point calculation elsewhere in the same layer to share hardware with — the INT8 version's main stage is now pure integer arithmetic, so that sharing opportunity no longer exists, and the exact same, unchanged adder simply gets counted in a different place instead. No new circuitry was actually added; +2 DSP of this increase is bookkeeping, not new hardware.
- **One small, genuinely new piece of hardware:** since the main calculation now produces an integer result but the final probability-normalization step still needs a real-world (floating-point) value to work with, one small multiplier is needed to bridge between the two representations — a real, if minor, addition (+1 DSP) that is an unavoidable and reasonable cost of the deliberate design choice explained above (keeping the final normalization step in floating-point, since it runs only once per classification).

In short: this is not evidence that combining INT8 and floating-point genuinely doubled the work in this layer — it is mostly a bookkeeping side-effect of removing floating-point arithmetic from elsewhere in the same layer, plus one small, deliberate, and justified new piece of hardware. It does not change the overall conclusion, since every other metric for this layer still improved substantially.

Headline Findings

- **Every single layer's memory/logic usage (LUT, FF) dropped substantially** — typically by a third to three-quarters — simply from using smaller 8-bit numbers instead of 32-bit ones.
- **Five of seven layers had failed to meet the required 100MHz clock speed target in Float32.** All five of those layers **passed** once converted to INT8. This is arguably the single most important hardware finding: **the Float32 design would not have reliably run at its intended speed; the INT8 design does.**
- **Most layers also got measurably faster** (lower latency), not just smaller — a result that is not automatic or guaranteed when quantizing, and is a genuine bonus finding.

(Source: Results/hls_results.md — every number above is drawn from real Vitis HLS csynth.rpt synthesis reports, generated by actually running the HLS tool on both versions of each layer, not estimated.)

Combined Conclusion — Why We Chose INT8

Putting both pieces of evidence together:

1. **Software evidence** shows INT8 costs **zero measured accuracy** compared to Float32, while needing only a quarter of the memory to store the model's weights.
2. **Hardware evidence** shows INT8 makes the real, synthesized circuit **smaller** (less memory/logic used), and — critically — **fixes real timing failures** that the Float32 version had in five of the seven layers, meaning the Float32 design would not have reliably worked at our required clock speed at all.

INT8 was not a compromise made to save resources at the cost of accuracy or correctness — in this project's specific case, it was a strict improvement on every measurable front: same accuracy, smaller hardware, and — unlike the Float32 version — hardware that actually meets its timing requirement.

INT4 was considered and rejected for the final implementation specifically because, unlike the jump from Float32 to INT8, going further to INT4 does cost measurable accuracy (97.80% vs 98.40%) for comparatively modest additional memory savings — a trade-off that was not judged worthwhile once INT8 had already achieved the necessary resource and timing improvements.

Part 3 — The Real, Complete Result: Two Full Bitstreams

Everything above was building toward one thing: a complete, working FPGA implementation of all seven network layers combined, targeting the actual PYNQ-Z2 board. **Both a Float32 version and an INT8 version of this complete network have now been built, verified for correctness against real data, and carried all the way through to a real, physically valid bitstream file** — the final output that would configure real FPGA hardware, if loaded onto a physical board.

Final, Real, Whole-System Comparison (from actual Vivado place-and-route, not estimates)

Metric	Float32 (complete network)	INT8 (complete network)	Change
Timing met?	Yes (+0.033 ns margin)	Yes (+0.121 ns margin — more comfortable)	Improved
General logic (LUT)	55.3% of chip	31.8% of chip	42.5% less
Memory bits (FF)	38.6% of chip	21.2% of chip	45.1% less

On-chip memory (BRAM)	85.4% of chip	25.7% of chip	69.9% less
Multiply-hardware (DSP)	10.9% of chip	20.9% of chip	Higher (expected — see Part 2's DSP explanation; still comfortably within budget)

Both versions work. Both meet timing with zero failing paths. The INT8 version uses dramatically less of the chip's logic, memory, and on-chip storage — while also running with a *more* comfortable timing margin than the Float32 version, not a worse one. The only resource that increased (DSP) was expected, precisely explained (Part 2, and in full technical detail in the main project document), and remains well within budget either way (20.9% used, out of 100% available).

The One-Sentence Version

We chose INT8 because our own evidence — both a software accuracy simulation and two real, complete FPGA bitstreams — shows it costs no accuracy, uses roughly a third to a half of the hardware resources of the Float32 version, and results in a design that meets its timing requirement with more margin, not less.